

Clase Básica: Crea una clase Vehiculo en Python con atributos marca y modelo, y un método descripcion() que retorne ambos.

Python

```
class Vehiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def descripcion(self):
        return f"Marca: {self.marca}, Modelo: {self.modelo}"
```

Instanciación: Crea dos objetos de Vehiculo (ej. Toyota y Ford) e imprime sus descripciones.

```
class Vehiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def descripcion(self):
        return f"Marca: {self.marca}, Modelo: {self.modelo}"

vehiculo1 = Vehiculo("Toyota", "Corolla")
vehiculo2 = Vehiculo("Ford", "Focus")

print(vehiculo1.descripcion())
print(vehiculo2.descripcion())
```

Salida esperada:

Marca: Toyota, Modelo: Corolla

Marca: Ford, Modelo: Focus

Uso de __init__: Crea una clase Usuario que requiera obligatoriamente un nombre y un email al ser creada.

```
class Usuario:
    def __init__(self, nombre, email):
```

```

        self.nombre = nombre
        self.email = email

    def mostrar_datos(self):
        return f"Nombre: {self.nombre}, Email: {self.email}"

usuario = Usuario("Juan Pérez", "juan@email.com")
print(usuario.mostrar_datos())

```

Simulación de Django: Si tuvieras un modelo en Django llamado Producto, ¿qué campos (atributos) le asignarías y qué métodos (def) podrías definir para gestionar el stock?

Si se desarrollara una aplicación de comercio electrónico en Django, el modelo Producto podría incluir los siguientes campos:

- nombre
- descripcion
- precio
- stock
- fecha_creacion

Además, podría definir métodos para gestionar el inventario.

```

from django.db import models

class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()
    precio = models.DecimalField(max_digits=10,
decimal_places=2)
    stock = models.IntegerField()
    fecha_creacion = models.DateTimeField(auto_now_add=True)

    def aumentar_stock(self, cantidad):
        self.stock += cantidad
        self.save()

    def reducir_stock(self, cantidad):
        if cantidad <= self.stock:

```

```
self.stock -= cantidad  
self.save()
```

```
def hay_stock(self):  
    return self.stock > 0
```